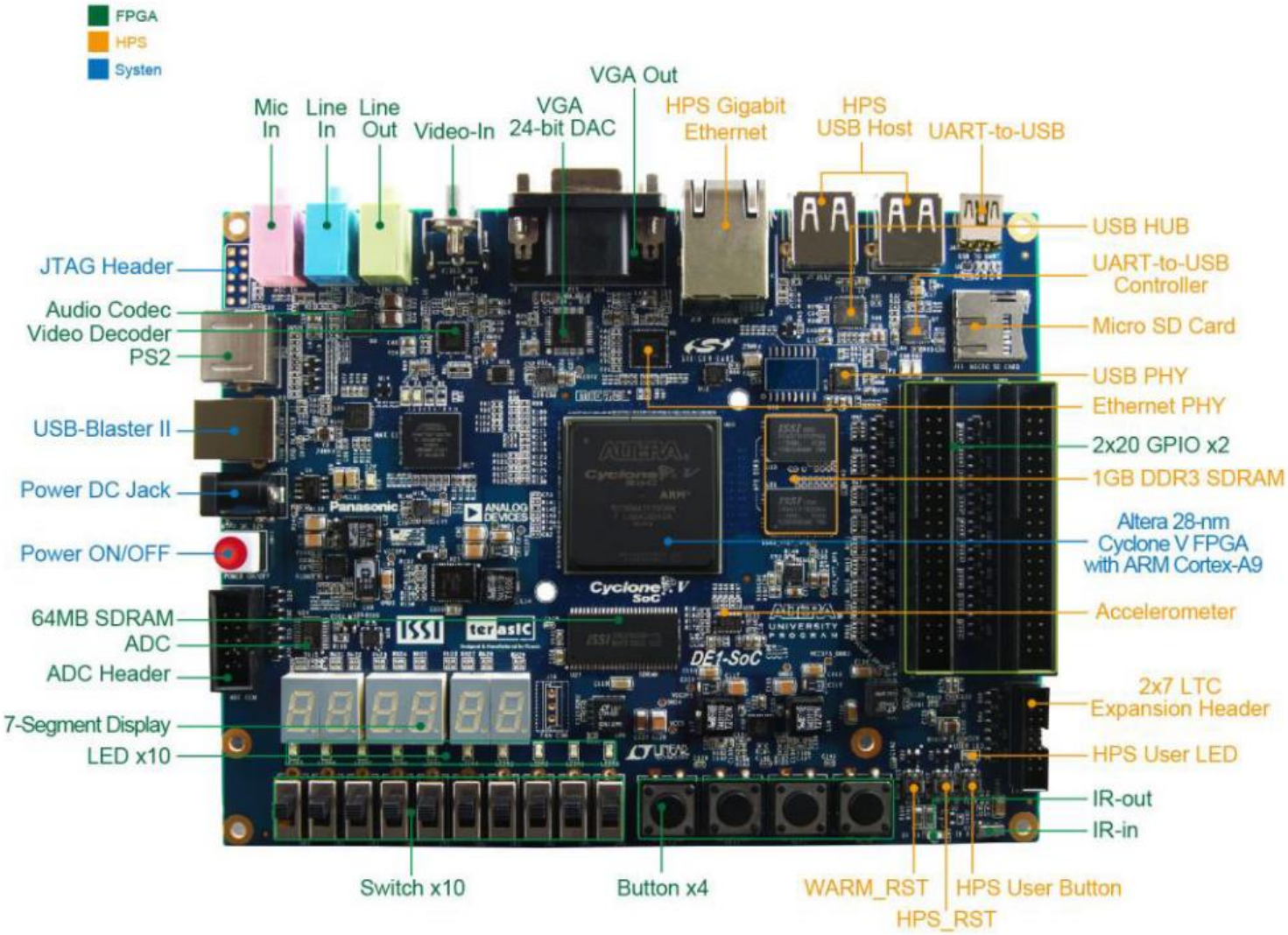
Chapter 4
Input/Output

4.1 Introduction of I/O devices in CPUlator

Objectives

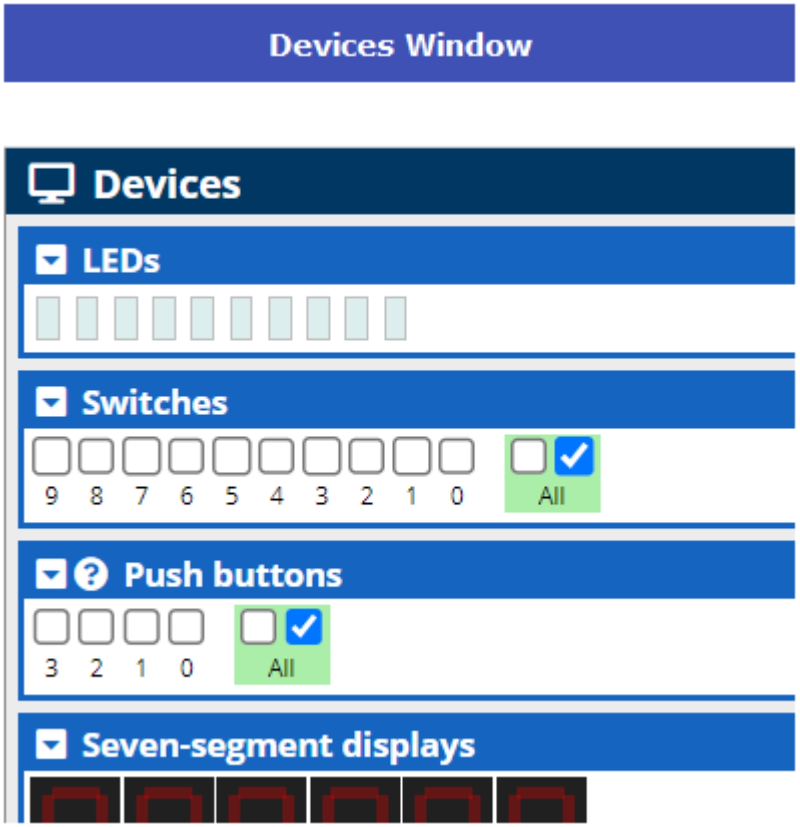
- Learn how to directly control an external device through assembly programs.
- External devices covered include:
  - LED
  - Button input
  - 7-segment LED
  - JTAG UART
  - Timer
- These devices are available in CPUlator as virtual input/output devices in the virtual De1-SoC computer board.

DE1-SoC


FPGA
HPS
Systen
Mic In
Line In
Line Out
Video-In
VGA 24-bit DAC
VGA Out
HPS Gigabit Ethernet
HPS USB Host
UART-to-USB
JTAG Header
Audio Codec
Video Decoder
PS2
USB-Blaster II
Power DC Jack
Power ON/OFF
64MB SDRAM
ADC
ADC Header
7-Segment Display
LED x10
Switch x10
Button x4
WARM_RST
HPS_RST
HPS User Button
USB HUB
UART-to-USB Controller
Micro SD Card
USB PHY
Ethernet PHY
2x20 GPIO x2
1GB DDR3 SDRAM
Altera 28-nm Cyclone V FPGA with ARM Cortex-A9
Accelerometer
2x7 LTC Expansion Header
HPS User LED
IR-out
IR-in

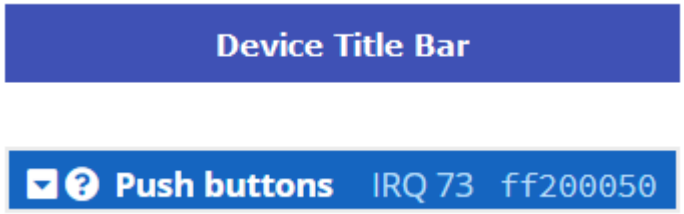
DE1-SoC Devices in CPUlator

- The simulated devices appear in the Devices window



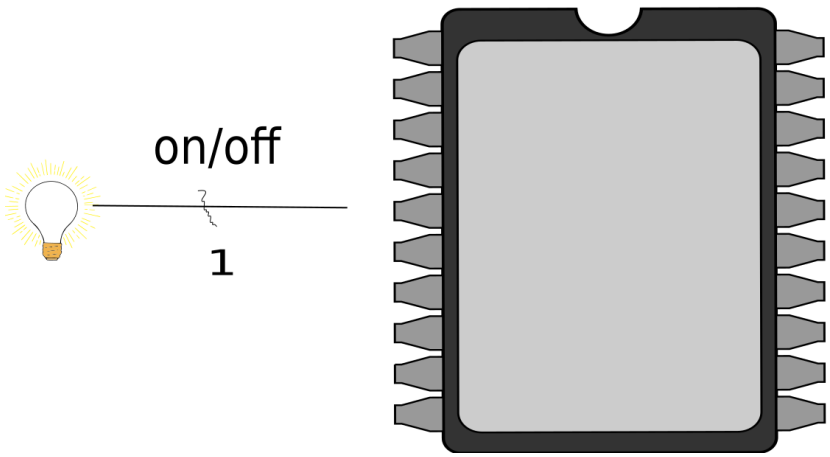
DE1-SoC Devices in CPUlator

- All individual devices panel have a title bar, like:



- All title bars have a device's hexadecimal base address (e.g., FF200050 above) which is used to read from or write to the device memory.
- Some title bars have an IRQ (interrupt request) value (e.g., IRQ 73 above) that is used to interface with that particular device.

The high-level picture



- From the main chip we want to control an (external) device, here an LED.
- We use one of the LEDs connected to the chip on the CPUlator.
- We want to send **1 bit** to this device to turn it **on/off**.

Low-level picture

Programmers need to:

- Identify the peripheral registers, in charge of LEDs, from the Altera De1-SoC manual:

https://ftp.intel.com/Public/Pub/fpgaup/pub/Intel_Material/18.1/Computer_Systems/DE1-SoC/DE1-SoC_Computer_ARM.pdf

- Identify the bits in the registers that control a specific LED
- Then, directly access the device via memory read/writes into these registers
- In the program, turning on an LED is typically just one write operation

Finding the registers in the manual

- The relevant information can be found under Section “2. De1-SoC Computer Contents” from the Altera De1-SoC manual
- Here, we need the information on the LEDs, in Section “Red LED Parallel Port”.
- Note that the base address is also available in the title bar in CPUlator:



LED9 …… LED0

LED explained

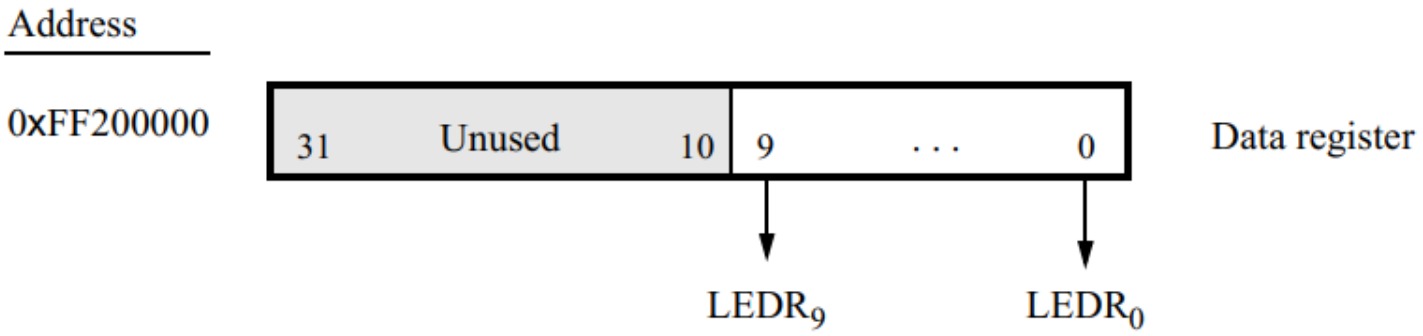


Figure 9. Output parallel port for *LEDR*.

The picture (and discussion in the manual) tells us

- The lowest 10 bits in the register at address 0xFF200000 are in charge of 10 LEDs hard-wired to the chip
- The remaining bits in the register are unused
- Note that the processor has a word-size of 32-bits
- Each of the lower bits controls one LED, and turns it on (bit: 1) or off (bit: 0)

Controlling an LED in ARM Assembler

```
1  // Constants
2  .equ LEDBASE, 0xff200000     // LED base address
3
4  .text  // Code Section
5  .global _start
6  _start:
7
8  ldr r0, =LEDBASE     // physical address of LEDs
9  mov r2, #0b00000001  // bitmask to turn on first LED
10 str r2, [r0]         // turn it on
```

- First, we load the bitmask 0b00000001 (first LED on, all other LEDs off) into register R2.
- Then we write this value into the LED register.
- Result: 

Blinking LED in ARM assembler

```
1  // Constants
2  .equ LEDBASE, 0xff200000     // LED base address
3
4  .text  // Code Section
5  .global _start
6  _start:
7  ldr r2, =LEDBASE @ physical address of the LED
8
9  loop:
10 eor r3, r3, #1 // toggle current value: 1->0, 0->1
11 str R3, [R2] // store value into LED register
12 bl delay // a delay function depending on the blinking frequency
13 b loop // infinite loop
```

Summary

- Controlling a simple external device means logically sending 1 bit of information (on/off)
- Realizing this control means physically writing into special registers which have special meaning
- The information on the special meaning is in the Altera De1-SoC manual
- Once understood, the code for direct device control is fairly short

Programming a Button input device

- The CPUlator simulates 4 push-buttons
- The push-buttons are used as input devices.
- To identify the registers involved, we need to look at “Pushbutton Key Parallel Port” of the De1-SoC Manual.

Push-button details

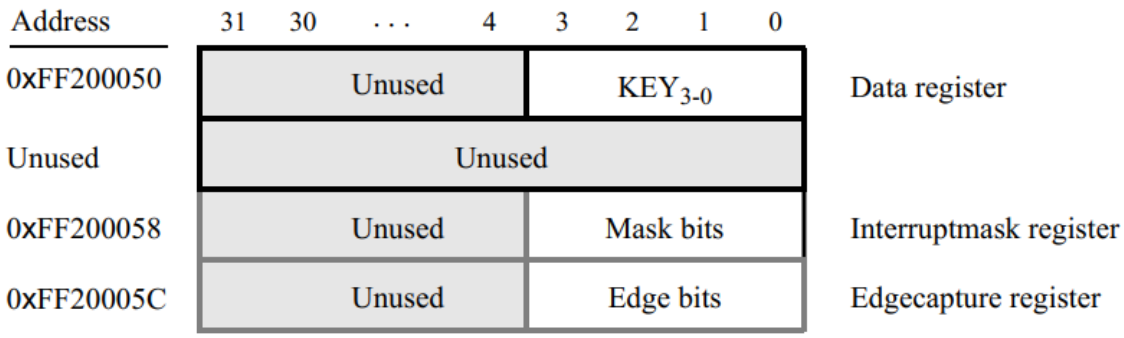


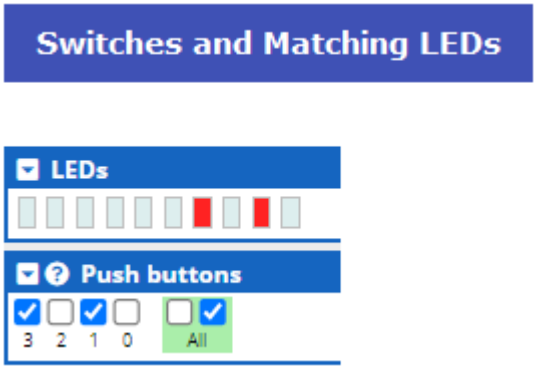
Figure 12. Registers used in the pushbutton parallel port.

- We need to read from address 0xFF200050 to determine the state of the buttons
- There is a one-to-one relationship between the lowest 4 bits in the register and the buttons:
  - if button 0 is pressed, bit 0 will be set to 1
  - if button 1 is pressed, bit 1 will be set to 1
- We will look at the remaining two registers later when we deal with interrupt.

Copying switches to LEDs

```
1  // Constants
2  .equ PUSHBASE,  0xff200050  // Push buttons base address
3  .equ LEDBASE,   0xff200000  // LEDS base address
4
5  .text  // Code Section
6  .global _start
7  _start:
8
9  ldr r0, =PUSHBASE
10 ldr r1, =LEDBASE
11
12 TOP:
13 ldr r2, [r0]    // Read in push buttons - active pressed
14 str r2, [r1]    // Copy to LEDs
15 b TOP
16
17 .end
```

The code above sets LEDs based on the buttons pressed:



🧠 Micro-challenges: LED

The **LED device register** is at address **0xFF200000**.

Which LEDs will turn on by the following code?

```
LDR r1, =0xFF200000      // LED register
MOV r2, #0b00001010
STR r2, [r1]
```

A. LED0 and LED2
B. LED1 and LED3
C. LED2 and LED3
D. LED0 and LED3

🧠 Micro-challenges: LED

- The following code reads the **LED register** and modifies it.
- What does this code do?

```
LDR r1, =0xFF200000
LDR r2, [r1]
EOR r2, r2, #0b1111111111
STR r2, [r1]
```

A. Turn on all LEDs
B. Toggle LED0
C. Toggle all LEDs
D. Turn off LED0

7-segment LEDs

- The CPUlator provides several 7-segment displays.
- The principle to program the display is the same as for the LED
- These displays can be used to display numbers, and some letters
- Therefore, the output is more meaningful than just one LED
- Relevant information is in “7-Segment Displays Parallel Port” of the manual.

7-segment LEDs

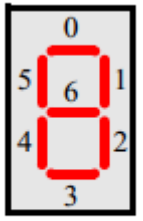
- 6 red 7-segment display



- The individual displays are labelled as HEX0 (rightmost) to HEX5 (leftmost).

7-segment LEDs

Address	Bits							
0xFF00020	31	30 - 24	23	22 - 16	15	14 - 8	7	6 - 0
		HEX3		HEX2		HEX1		HEX0
0xFF00030	31	30 - 24	23	22 - 16	15	14 - 8	7	6 - 0
		HEX7		HEX6		HEX5		HEX4



Segments

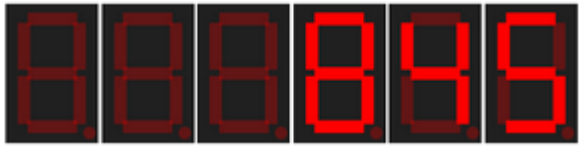
- The 7 segments in the HEX display are each controlled by 1 bit in the HEX data register
- The mapping of the bit positions to the segments is shown in the picture on the right, e.g., bit 0 controls the upper bar in display 1
- We need 7 bits per HEX display in the register (8th bit unused).

Example

Write the digits 845 to the rightmost three 7-segment displays (HEX0 to HEX2):

```
1  // Constants
2  .equ SEVEN_SEGMENT_BASE, 0xff200020 // first four 7-Segment Displays base address
3  // Define digit display bit offsets
4  .equ HEX1,  8
5  .equ HEX2,  16
6  // Define block digit segments
7  .equ FOUR,  0b01100110
8  .equ FIVE,  0b01101101
9  .equ EIGHT, 0b01111111
10
11 .org 0x1000
12 .text
13 .global _start
14 _start:
15
16 ldr r0, =SEVEN_SEGMENT_BASE    // load first seven segment address
17 ldr r1, =FIVE                 // load segments definition of 5
18 ldr r2, =FOUR                 // load segments definition of 4
19 orr r1, r1, r2, lsl #HEX1     // or definitions of 5 and 4 - shift 4 to proper digit
20 ldr r2, =EIGHT                // load segments definition of 8
21 orr r1, r1, r2, lsl #HEX2     // or definitions of 5, 4, and 8 - shift 8 to proper digit
22 // equivalent to:
23 // ldr r1, =#0x007F666D
24 str r1, [r0]                  // write all digits to seven segment display
25
26 _stop:
27 b _stop
28 .end
```

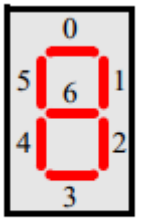
Result:



Micro-challenges: 7-segment LED

- The **7-segment display register** for **HEX3–HEX0** is at address **0xFF200020**.
- What will be displayed on the two rightmost 7-segment displays by the following code?

```
LDR r1, =0xFF200020
LDR r2, =0x00007D07
STR r2, [r1]
```



Segments

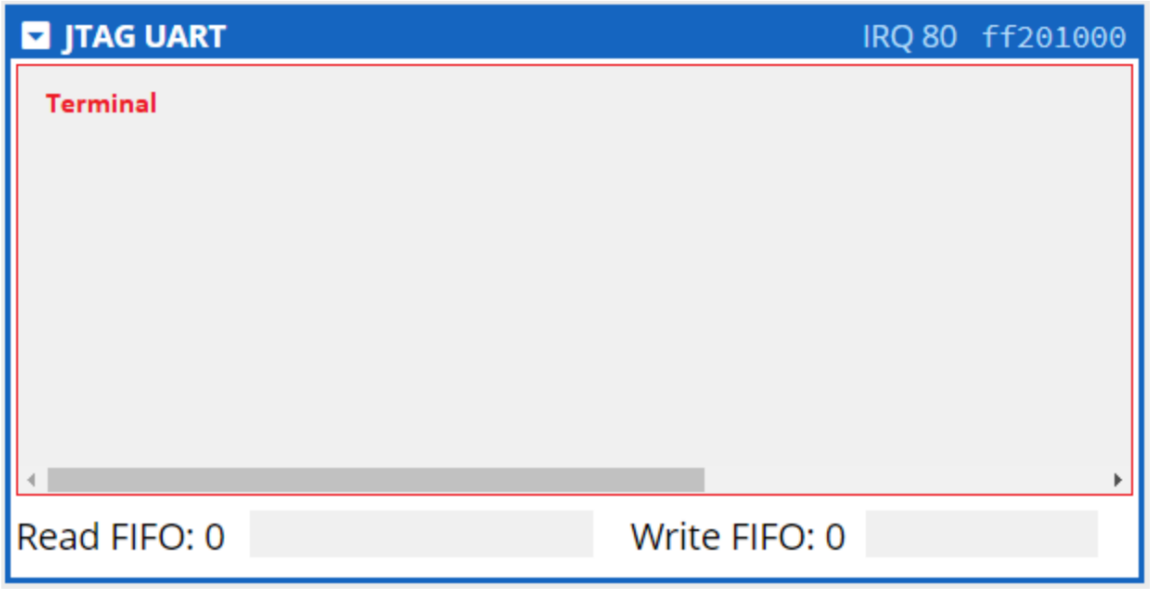
A. 67
B. 76
C. 09
D. 90

JTAG UART

- A ‘real’ DE-SoC computer can be connected to an external device (e.g., keyboard, monitor) and communicate to it through its JTAG UART.
- CPUlator cannot communicate to an external device, but it can simulate:
  - writing to
  - reading from

  a JTAG UART box.

JTAG UART Console



- Character input is handled by the 64-byte Read FIFO queue
- Character output by the 64-byte Write FIFO queue.

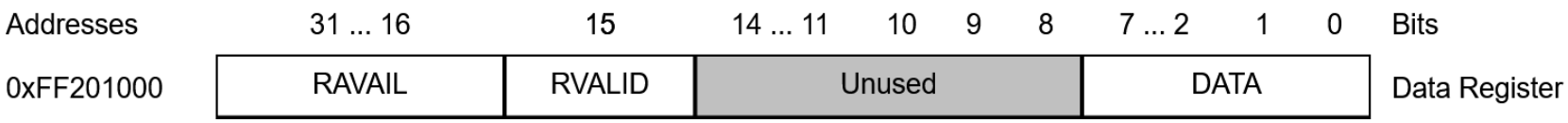
Read FIFO Queue

- Enter characters into the queue by clicking on the terminal area and typing in CPUlator.
- These characters then appear in the Read queue in hexadecimal format.

```
Read FIFO: 10 48 69 20 74 68 65 72 65 2e 0a
```

- Characters entered beyond the 64-byte limit are ignored and no more characters may be entered into the Read FIFO until the queue is *read by the microcontroller*.

UART Data Register



- Data: a character (byte) at the head of the queue.
- RAVAIL: Number of characters in the queue. RAVAIL decrements once after each read.
- RVAILD: 1 if RAVAIL > 0, otherwise 0.

Reading a string (Lab 8)

ReadString subroutine

```
39 ReadString:
40 /*
41 -----------------------------------------------------
42 Reads an ENTER terminated string from the UART.
43 -----------------------------------------------------
44 Parameters:
45   r4 - address of string buffer
46 Uses:
47   r0 - holds character to print
48   r1 - address of UART
49 -----------------------------------------------------
50 */
51
52 stmfd sp!, {r0, r1, r4}  // preserve temporary registers
53 ldr   r1, =UART_BASE     // get address of UART
54
55 rsLOOP:
56 ldr  r0, [r1]       // read the UART data register
57 tst  r0, #VALID     // check if there is new data
58 beq  _ReadString    // if no data, exit subroutine
59 strb r0, [r4], #1       // store the character in memory
60 b    rsLOOP
61 _ReadString:
62 ldmfd sp!, {r0, r1, r4}  // recover temporary registers
63 bx    lr                 // return from subroutine
```

Main Program

```
21 ldr    r4, =First
22 bl    ReadString
```

New String in Memory

Memory (Ctrl-M)

Go to address, label, or register: 1000 Refresh

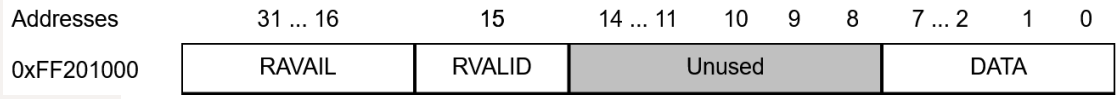
Address	Memory contents and ASCII				
00001000	2c 10 9f e5	2c 40 9f e5	50 50 84 e2	00 00 91 e5	,••• ,@•• PP•• ••••
00001010	02 29 10 e2	05 00 00 0a	ff 00 00 e2	00 00 c4 e5	•)•• •••• •••• ••••
00001020	01 40 84 e2	05 00 54 e1	00 00 00 0a	f6 ff ff ea	•@•• ••T• •••• ••••
00001030	fe ff ff ea	00 10 20 ff	40 10 00 00	00 00 00 00	•••• •• • @••• ••••
00001040	54 68 69 73	20 69 73 20	61 6e 20 69	6e 70 75 74	This is an i nput
00001050	20 73 74 72	69 6e 67 2e	00 00 00 00	00 00 00 00	str ing. •••• ••••
00001060	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	•••• •••• •••• ••••
00001070	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	•••• •••• •••• ••••
00001080	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	•••• •••• •••• ••••
00001090	aa aa aa aa	aa aa aa aa	aa aa aa aa	aa aa aa aa	•••• •••• •••• ••••

String buffer to store values read from UART

```
65 .data
66 .align
67 // The list of strings
68 First:
69 .space  SIZE
```

Reading a string (Lab 8)

```
1   .equ UART_BASE, 0xff201000 // UART base address
2   …
3   .equ VALID, 0x8000  // Valid data in UART mask
4   …
5   ldr  r1, =UART_BASE
6   …
7   ldr  r0, [r1]       // read the UART data register
8   tst  r0, #VALID     // check if there is new data
9   beq  _ReadString    // if no data, return 0
10  …
```

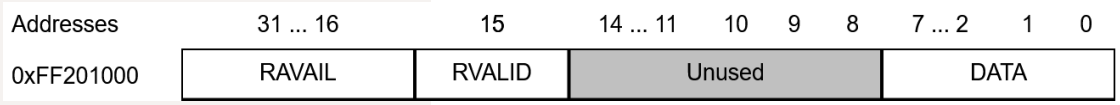


1. Load the contents of the UART data register into r0.
2. Note if there is new data to be read, RVALID (the 15th bit of r0) is set to 1.
   - `tst r0, #VALID` performs the "and" operation between r0 and the mask with only the 15th bit set to 1 (i.e., 0x8000)
   - if RVAILD = 0, the "and" operation result is zero, thereby setting the Z flag to 1. Otherwise Z = 0.
3. Use the `beq` branch instruction to branch to `_ReadString` if RVALID = 0 → no writing to the list is required. Otherwise, the character read will be written to the list.

Reading a string (Lab 8)

The actual characters in the *read FIFO* are copied into memory by the lines:

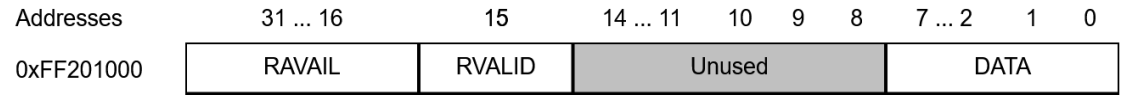
```
1  .equ UART_BASE, 0xff201000 // UART base address
2  …
3  ldr  r1, =UART_BASE
4  ldr  r4, =TEXT_strING
5  …
6  strb r0, [r4], #1  // store the character in memory
7                     // move to next byte in storage buffer
8  …
```



- r0 contains the whole data register. Only the last byte is data. Thus, `strb` is used to copy the data to the text string in memory.
- move to the next byte location to prepare for writing the next character.

Micro-challenges: UART

- The **UART data register** is at **0xFF201000**.

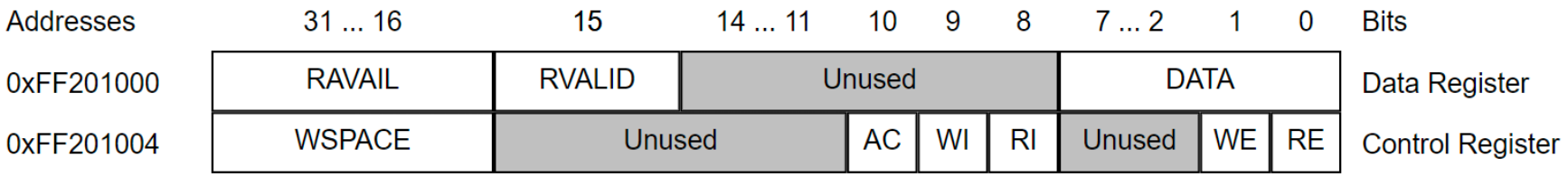


- What is this code checking?

```
LDR r1, =0xFF201000
LDR r0, [r1]
TST r0, #0x8000
BEQ stop
```

A. Whether UART transmit buffer is full
B. Whether a character is available to read
C. Whether UART is enabled
D. Whether FIFO overflow occurred

Write FIFO queue (Lab 8)



- To write a character to UART, write to the least significant byte of the Data register.
- The character is then written to the Write FIFO queue.
- Characters in the Write FIFO are automatically removed from the queue and displayed in the Terminal over time.
- The WSPACE (remaining space available in the queue) field in the control register is then decremented.
- Write FIFO queue stores 64 bytes. If the Write FIFO queue is full, the character written to the Data register will be lost.

Writing a string (Lab 8)

WriteString subroutine

```
33 WriteString:
34 /*
35 -----------------------------------------------------
36 Prints a null terminated string to the UART.
37 -----------------------------------------------------
38 Parameters:
39   r4 - address of string to print
40 Uses:
41   r0 - holds character to print
42   r1 - address of UART
43 -----------------------------------------------------
44 */
45 stmfd sp!, {r0, r1, r4}  // preserve temporary registers
46 ldr   r1, =UART_BASE     // get address of UART
47
48 wsLOOP:
49 ldrb  r0, [r4], #1       // load a single byte from the string
50 cmp   r0, #0
51 beq   _WriteString       // stop when the null character is found
52 strb   r0, [r1]           // copy the character to the UART DATA field
53 b     wsLOOP
54 _WriteString:
55 ldmfd sp!, {r0, r1, r4}  // recover temporary registers
56 bx    lr                 // return from subroutine
```

Main program

```
18 ldr r4, =First
19 bl  WriteString
20 ldr r4, =Second
21 bl  WriteString
22 ldr r4, =Third
23 bl  WriteString
24 ldr r4, =Last
25 bl  WriteString
26
27 _stop:
28 b    _stop
```

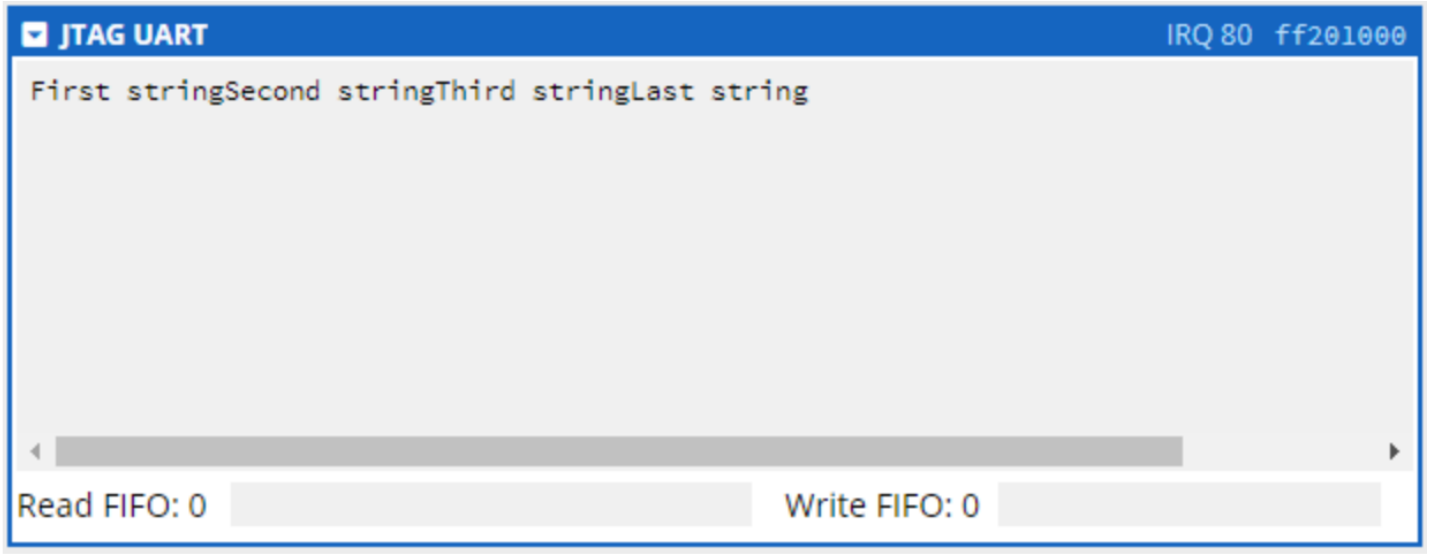
Data strings

```
58 .data
59 .align
60 // The list of strings
61 First:
62 .asciz  "First string"
63 Second:
64 .asciz  "Second string"
65 Third:
66 .asciz  "Third string"
67 Last:
68 .asciz  "Last string"
69 _Last:    // End of list address
```

The `.asciz` directive defines a null-terminated string.

Writing a string (Lab 8)

Here is the result of the program:



🧠 Micro-challenges: UART

- The following code reads characters from a **string in memory** and writes them to the **UART data register**.
- LDR r1, =0xFF201000 // UART data register

```
loop:
LDRB r0, [r4], #1
CMP r0, #0
BEQ done
STRB r0, [r1]
B loop
```

What stops the loop?

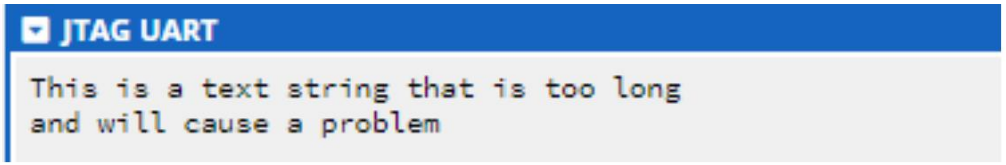
A. UART FIFO becomes empty
B. End of string (NULL character)
C. FIFO becomes full
D. The memory pointer reaches the end of the UART register

Overflow in Writing to a String

- ARM writes to the Write FIFO queue very quickly, quicker than the rate characters can be dumped to the terminal.
- There is a chance that the Write FIFO would be full if we write a long string.
- e.g., Writing this string with the previous program

```
TEST_STRING:
.asciz    "This is a text string that is too long\nand will cause a problem by overflowing the Write FIFO.\n"
.end
```

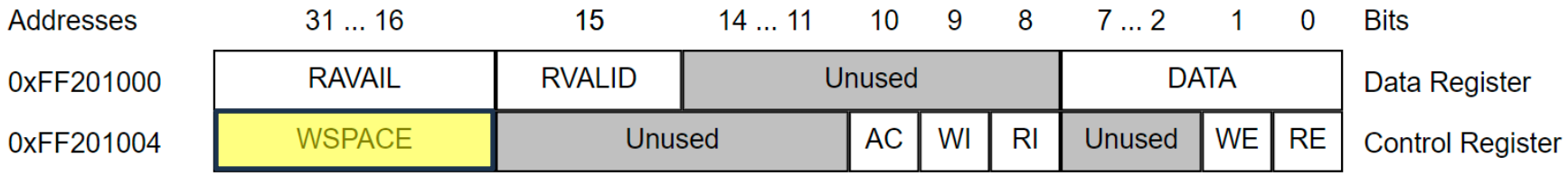
produces this output only



and writes the following error to the Messages dialog

UART at ff201000 write FIFO overflowed when trying to write 62. Did you check for available write space first?
Simulator requested a breakpoint.

Overflow in Writing to a String



- Before writing to DATA, make sure that WSPACE > 0

If WSPACE = 0

```
48 stmfd sp!, {r0, r1, r2, r4, r5}  // preserve temporary registers
49 ldr  r1, =UART_BASE     // get address of UART
50 ldr  r2, =WSPACE_MASK   // assign WSPACE_MASK = 0xffff0000 to a register -
51                         // too large to use as immediate value
52
53 wsLOOP:
54 ldrb  r0, [r4], #1       // load a single byte from the string
55 cmp   r0, #0
56 beq   _WriteString       // stop when the null character is found
57
58 WSPACE:             // verify that the Write FIFO has space
59 ldr  r5, [r1, #4]   // load Control Register
60 tst  r5, r2         // tst performs AND operation to isolate WSPACE field from
61                     // control register 0xff201004
62 beq  WSPACE         // recheck WSPACE field until it is non-zero
63
64 strb   r0, [r1]           // copy the character to the UART DATA field
65 b     wsLOOP
66
67 _WriteString:
68 ldmfd sp!, {r0, r1, r2, r4, r5}  // recover temporary registers
69 bx    lr                 // return from subroutine
```

Micro-challenges: UART

- The following code writes characters from a string in memory to the **UART data register**. Assume:
- r1 = 0xFF201000 is the **UART data register base address**
- WSPACE_MASK = 0xFFFF0000

```
ldr   r1, =0xFF201000
ldr   r2, =WSPACE_MASK      // WSPACE_MASK = 0xFFFF0000

wsLOOP:
ldrb  r0, [r4], #1          // load one character from the string
cmp   r0, #0
beq   done                  // stop at null character

WSPACE:
ldr   r5, [r1, #4]
tst   r5, r2
beq   WSPACE

strb  r0, [r1]
b     wsLOOP

done:
```

Addresses	31 ... 16	15	14 ... 11	10	9	8	7 ... 2	1	0	Bits
0xFF201000	RAVAIL	RVALID	Unused				DATA			Data Register
0xFF201004	WSPACE	Unused		AC	WI	RI	Unused	WE	RE	Control Register

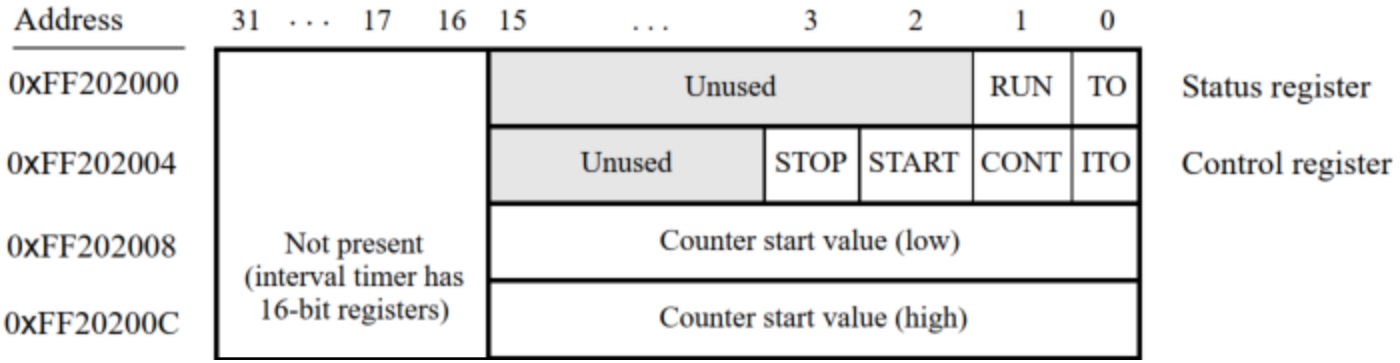
What is the purpose of the highlighted fragment in this program?

A. It checks whether a character is available to read from UART
B. It waits until there is space in the UART Write FIFO before writing
C. It clears the UART control register before each character is written
D. It detects the end of the string in memory

Timer

- Counter is a register that can be loaded with a binary number (count) which can be decremented per clock cycle.
- We can use this counter to measure time.
- Analogous to setting your iPhone timer to a specific time (e.g., 3 minutes) and an alarm will be sounded after the 3 minutes have elapsed.
- Will introduce the ARM A9 MPCore Private Timer and the Interval Timer here.

Interval Timer



- Users write an initial count value to the Counter start value register. This controls the time delay generated by the timer.
- **TO**: Set to 1 by the timer when it has reached 0. Reset by writing a 0 into it.
- **START/STOP**: commence/suspend the operation of the timer by writing a 1 to the respective bit.
- **ITO**: Enable interrupt
- **CONT**: If CONT = 1, when the timer reaches 0, it automatically reloads the count value and the timer continues counting down. If CONT = 0, the timer stops after it has reached 0.

Interval Timer

- Clock frequency = 100MHz
- Clock period = 1/100MHz = $\frac{1}{100}\mu s$
- How should we initialize the Count start value register to generate a specific time delay?
- Time delay = Clock period x Count start value
- Count start value = Time delay / clock period
- e.g., Time delay = 1 s

Count start value = $\frac{1}{\frac{1}{100}} \times 10^6 = 100 \times 10^6$

Example

This program toggles the rightmost LED every 1 second:

```
1  .equ TIMER_BASE, 0xff202000  // Interval Timer base addresss
2  .equ LED_BASE, 0xff200000 // LED Base
3
4  .text
5  .global _start
6  _start:
7
8  LDR r0, =TIMER_BASE
9  ldr r3, =LED_BASE
10 mov r4, #0
11 str r4, [r3]
12
13 /* set the interval timer period for scrolling the led displays */
14 ldr r1, =100000000
15 // 1/(100 mhz) x 100x10^6 = 1 sec
16 str r1, [r0, #0x8] // store the low half word of counter
17 // start value
18 lsr r1, r1, #16
19 str r1, [r0, #0xc] // high half word of counter start value
20
21 // start the interval timer, enable its interrupts
22 mov r2, #0 // initialize T0 to be 0
23 str r2, [r0]
24
25 mov r1, #0x7 // START = 1, CONT = 1, ITO = 1
26 str r1, [r0, #0x4]
27
28 loop:
29 wait:
30 ldr r1, [r0] // read T0: T0 would set to 1 if the timer counts to 0
31 tst r1, #0b1
32 beq wait
33 str r2, [r0] // re-intialize T0 to be 0
34 eor r4, r4, #1
35 str r4, [r3]  // toggle LED
36 b loop
37
38 .end
```

Address	31 ··· 17	16	15 ... 3	2	1	0	
0xFF202000	Not present (interval timer has 16-bit registers)		Unused		RUN	TO	Status register
0xFF202004			Unused	STOP START	CONT	ITO	Control register
0xFF202008			Counter start value (low)				
0xFF20200C			Counter start value (high)				

ARM A9 Private Timer

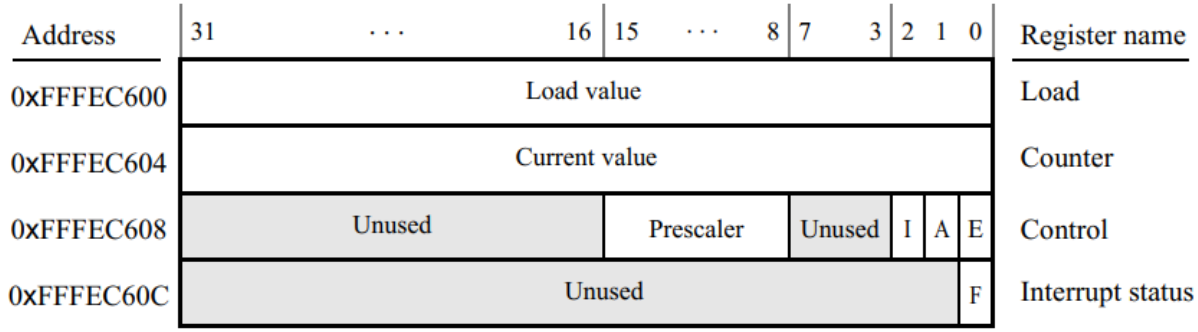


Figure 3. ARM A9 private timer port.

- Users first write an initial count value to the *Load* register. This controls the time delay generated by the timer.
- The timer is started by setting the enable bit E to 1 and it can be stopped by setting E to 0.
- Once enabled, the timer decrements until it reaches 0.
- When it reaches 0:
  - the timer sets F bit in the *Interrupt Status* register, indicating the desired time period has been elapsed. F bit is reset to 0 by writing a 1 to it.
  - the timer will stop if the auto bit A in the *Control* register is set to 0. Otherwise, the timer will automatically reload the value in the Load register and continue decrementing.

ARM A9 Private Timer

- Clock frequency = 200MHz
- Clock period = 1/200MHz = $\frac{1}{200}\mu s$
- How should we initialize the Load register to generate a specific time delay?
- Time delay = Clock period x Load
- Load = Time delay / clock period
- e.g., Time delay = 1 s

$$\text{Load} = \frac{1}{\frac{1}{200}} \times 10^6 = 200 \times 10^6$$

ARM A9 Private Timer

- The *Prescaler* in the *Control* register slows down the counting rate, thereby extending the delay time period.
- The timer decrements every *Prescaler* + 1 clock cycles
  - Prescaler = 0: decrements every clock cycle
  - Prescaler = 1: decrements every two clock cycles.
- Time delay = Clock period x Load x (Prescaler + 1)
- With the same initial Load register value, the time delay is extended by a factor of Prescaler + 1.
- e.g., Load = 200×10^6:
  - Prescaler = 0, time delay = 1s
  - Prescaler = 1, time delay = 2s …..
  - Prescaler = 255, time delay = 256s

Example

This program toggles the rightmost LED every 1 second:

```
1  // Constants
2  .equ PRIVATE_TIMER_BASE, 0xFFFEC600 // Private Timer Base
3  .equ LED_BASE, 0xFF200000 // LED Base
4
5  .org 0x1000
6  .text
7  .global _start
8  _start:
9
10 ldr r0, =LED_BASE
11 mov r2, #0
12 str r2, [r0]
13
14 ldr r1, =PRIVATE_TIMER_BASE // MPCore private timer base address
15 mov r4, #1
16 str r4, [r1, #0xC] //Writing 1 to the interrupt status bit resets it
17
18 ldr r3, =200000000 // timeout = 1/(200 MHz) x 200 x 10^6 = 1 sec
19 str r3, [r1] // write to timer load register
20 mov r3, #0b011 // set bits: mode = 1 (auto), enable = 1
21 str r3, [r1, #0x8] // write to timer control register
22 LOOP:
23 WAIT: ldr r3, [r1, #0xC] // read timer status
24 cmp r3, #0
25 beq WAIT // wait for timer to expire
26 str r4, [r1, #0xC] // reset timer flag bit
27 eor r2, r2, #1 // toggle LEDG value
28 str r2, [r0]
29 b LOOP
30 .end          // write all digits to seven segment display
```

Address	31 ··· 16	15 ··· 8	7 3	2	1	0	Register name
0xFFFEC600	Load value						Load
0xFFFEC604	Current value						Counter
0xFFFEC608	Unused	Prescaler	Unused	I	A	E	Control
0xFFFEC60C	Unused					F	Interrupt status

🧠 Micro-challenges: Timer

You want to use the ARM A9 Private Timer to generate a time delay of **exactly** 20 seconds. The frequency of the timer is 200MHz. For each Load value listed in the table, what value should be used as the Prescaler? Note that the Load values are expressed as decimal values.

Load	Prescaler
10^9	
2×10^8	
10^8	
5×10^7	